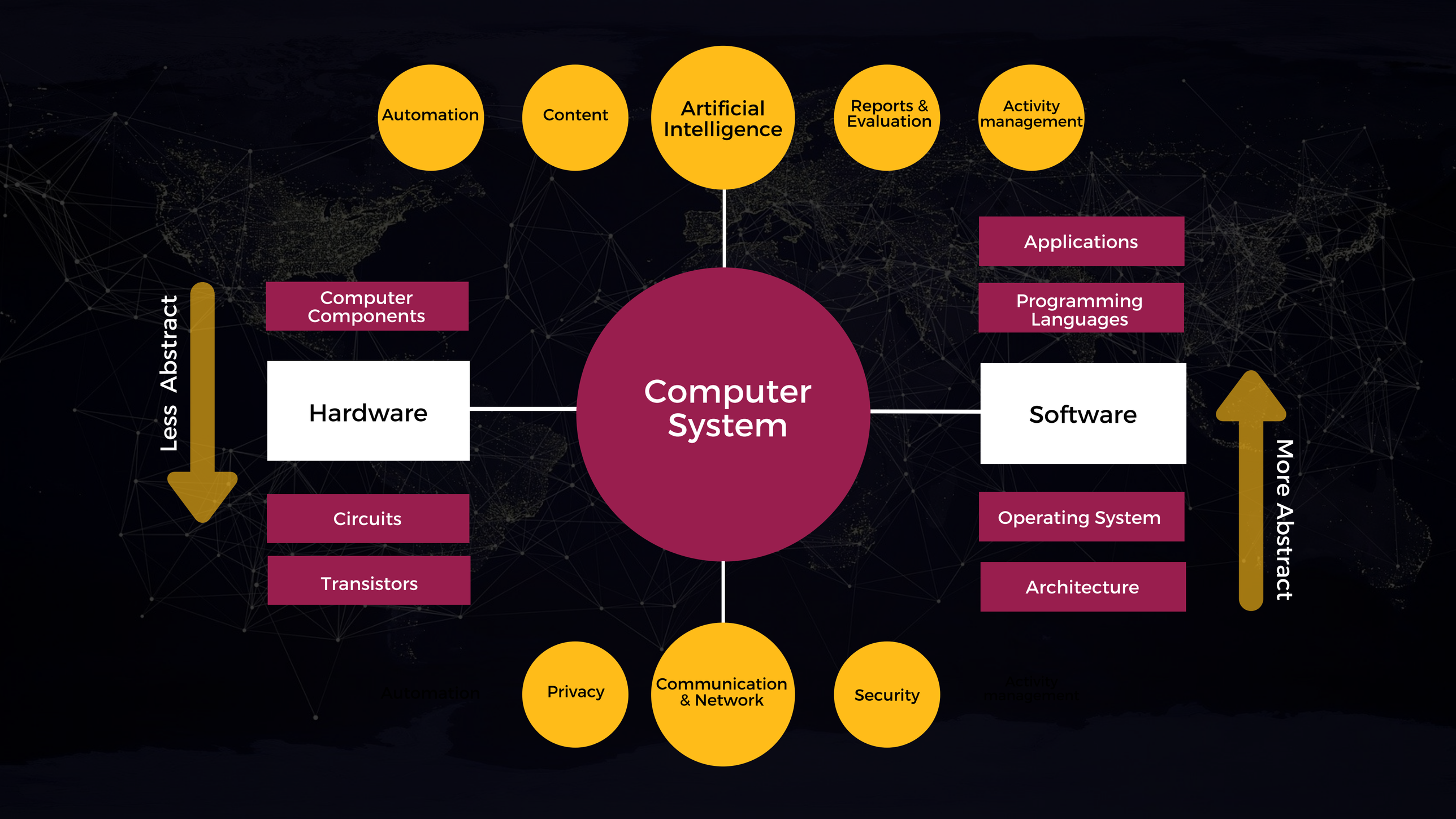
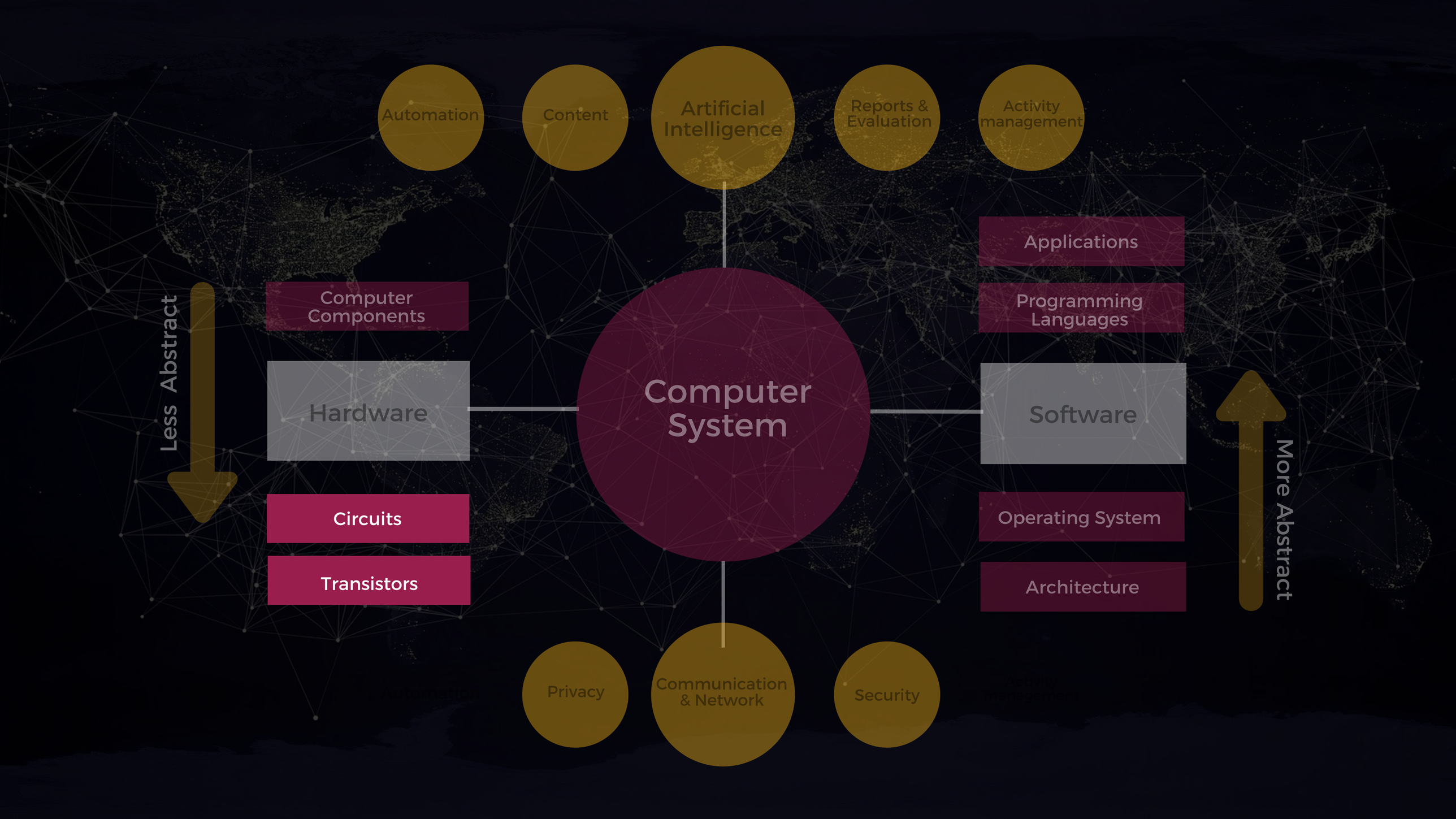


LOYOLA UNIVERSITY CHICAGO

Introduction to Computing

MODULE 3: DIGITAL LOGIC AND COMPUTER HARDWARE


Automation
Content
Artificial Intelligence
Reports & Evaluation
Activity management
Computer Components
Hardware
Circuits
Transistors
Less Abstract
Computer System
Applications
Programming Languages
Software
Operating System
Architecture
More Abstract
Privacy
Communication & Network
Security


Automation
Content
Artificial Intelligence
Reports & Evaluation
Activity management
Computer Components
Hardware
Circuits
Transistors
Less Abstract
Computer System
Applications
Programming Languages
Software
Operating System
Architecture
More Abstract
Privacy
Communication & Network
Security

BOOLEAN ALGEBRA AND TRUTH TABLES

Boolean Algebra, Truth Tables, and Digital Circuits

- Recall boolean expressions in Python:
  - Logic operations AND, OR, and NOT.
- Algebraic notation:
  - For Truth table, we will use 1 and 0 for True and False.
  - **A or B** is written with a + sign: **A + B**
  - **A and B** is written as with mathematical notation for multiplication: **AB**
  - **not A** is written: **A'**
- Because there are a few possible input values, we can build a **truth table** to get the results of all possible combinations.

TRUTH TABLE FOR OR

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

- The output is **0** only if all inputs are **0**

TRUTH TABLE FOR AND

A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1

- The output is 1 only if all inputs are 1

TRUTH TABLE FOR NOT

A	A’
0	1
1	0

- The output is generated by reversing the input.
- Python assume NOT has highest precedence followed by AND, and then OR last.

Exercise 1

- Write a Python program that produces the truth table for:
  A. **OR**
  B. **AND**
  C. **NOT**

Exercise 1

For A OR B:

```
for A in [0, 1]:
    for B in [0, 1]:
        print("{} or {} = {}".format(A, B, A or B))
```

```
0 or 0 = 0
0 or 1 = 1
1 or 0 = 1
1 or 1 = 1
```

TRUTH TABLE FOR XOR

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

- The XOR is short for eXclusive OR.
- XOR is true when exactly one of the two inputs is true.
- It is logically a variation on OR, so rather than a plain '+', it has a circle around it: ⊕

Exercise 2

Write a Python program that produces the truth table XOR:

1

```
for A in [0, 1]:
    for B in [0, 1]:
        print("{} xor {} = {}".format(A, B, A ^ B))
```

```
0 xor 0 = 0
0 xor 1 = 1
1 xor 0 = 1
1 xor 1 = 0
```

Exercise 2

Write a Python program that produces the truth table XOR:

2

```
for A in [0, 1]:
    for B in [0, 1]:
        print("{} xor {} = {}".format(A, B, int(A != B)))
```

```
0 xor 0 = 0
0 xor 1 = 1
1 xor 0 = 1
1 xor 1 = 0
```

Exercise 2

Write a Python program that produces the truth table XOR:

3

```
for A in [0, 1]:
    for B in [0, 1]:
        print("{} xor {} = {}".format(A, B, int((A and not B) or (not A and B))))
```

```
0 xor 0 = 0
0 xor 1 = 1
1 xor 0 = 1
1 xor 1 = 0
```

Truth Tables for Complicated Expressions

- Think of the last code (solution number 3) for solving the XOR.

A	B	B’	AB’
0	0	1	0
0	1	0	0
1	0	1	1
1	1	0	0

A	A’	B	A’B
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0



AB’	A’B	AB’+A’B
0	0	0
0	1	1
1	0	1
0	0	0

Truth Tables for Complicated Expressions

- Let's have another example: **AB +AC**

A	B	C	AB	AC	AB+AC
0	0	0	0	0	**0**
0	0	1	0	0	**0**
0	1	0	0	0	**0**
0	1	1	0	0	**0**
1	0	0	0	0	**0**
1	0	1	0	1	**1**
1	1	0	1	0	**1**
1	1	1	1	1	**1**

Truth Tables for Complicated Expressions

- We can also use A(B+C), since **AB + AC = A(B+C)** using the distributive property.

A	B	C	AB	AC	AB+AC
0	0	0	0	0	**0**
0	0	1	0	0	**0**
0	1	0	0	0	**0**
0	1	1	0	0	**0**
1	0	0	0	0	**0**
1	0	1	0	1	**1**
1	1	0	1	0	**1**
1	1	1	1	1	**1**

A	B	C	B+C	A(B+C)
0	0	0	0	**0**
0	0	1	1	**0**
0	1	0	1	**0**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	1	1	**1**
1	1	0	1	**1**
1	1	1	1	**1**

Exercise 3

Write a Python program that produces the truth table AB + AC

05 MINUTES



Exercise 3

Write a Python program that produces the truth table AB + AC:

```
for A in [0, 1]:
    for B in [0, 1]:
        for C in [0, 1]:
            print("{0}x{1} + {1}x{2} = {3}".format(A, B, C, int(A and B or A and C)))
```

Rules of Boolean Algebra

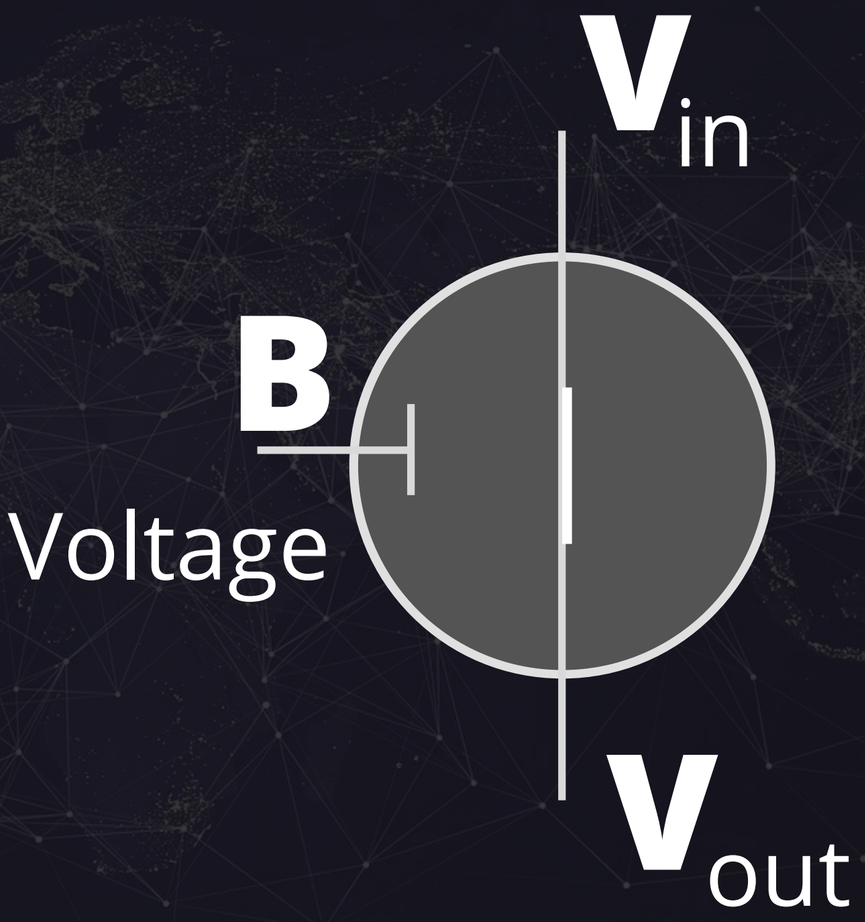
Property name	AND version	OR version
Commutative	AB = BA	A + B = B + A
Associative	(AB)C = A(BC)	(A+B)+C = A+(B+C)
Distributive	AB+AC = A(B+C)	A+BC = (A+B)(A+C)
Identity	1A = A	0 + A = A
Complement	(A')' = A	A + A' = 1
DeMorgan's Law	(AB)' = A' + B'	(A+B)' = A' B'

TRANSISTORS, GATES, AND CIRCUITS

Transistors

- A transistor is a semiconductor device used to amplify or switch electronic signals and electrical power.
- Transistors are used in **digital circuits** as switches to signal "on" or "off" state.
  - High-power applications: Switched-mode power supplies
  - Low-power applications: Logic gates.

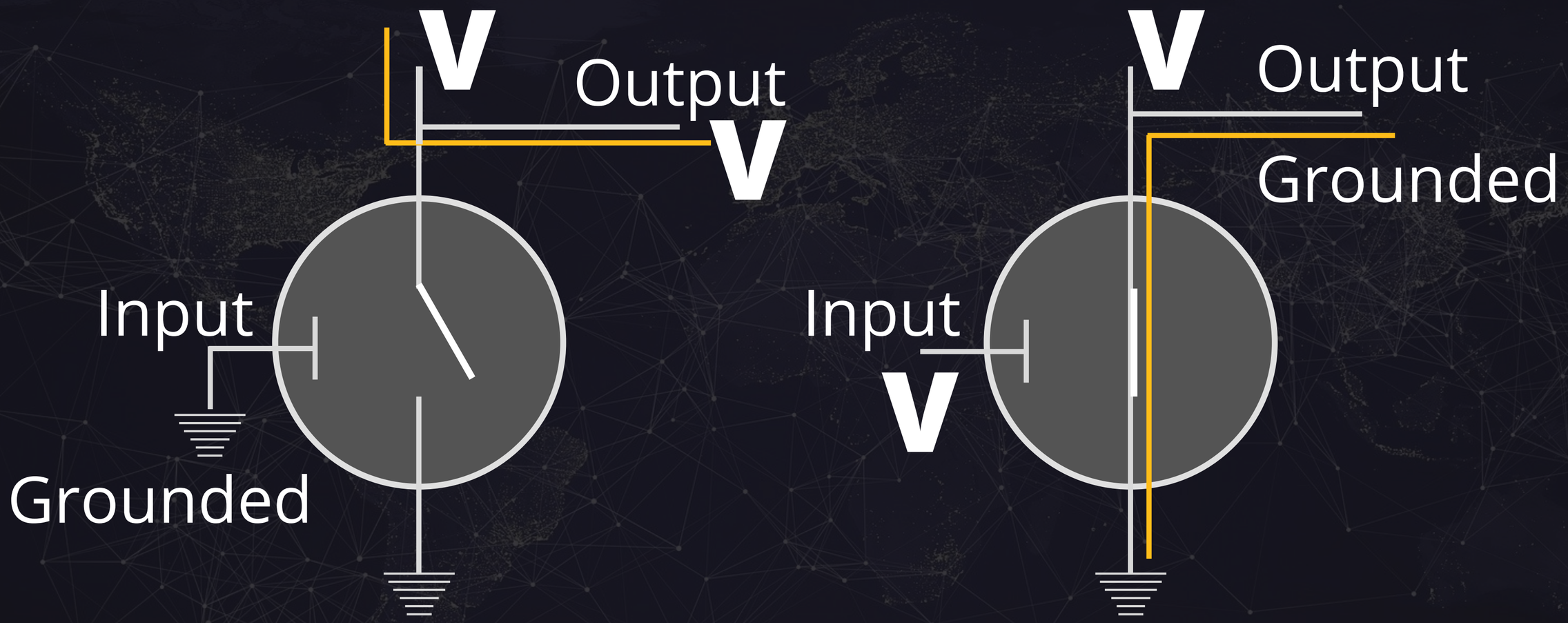




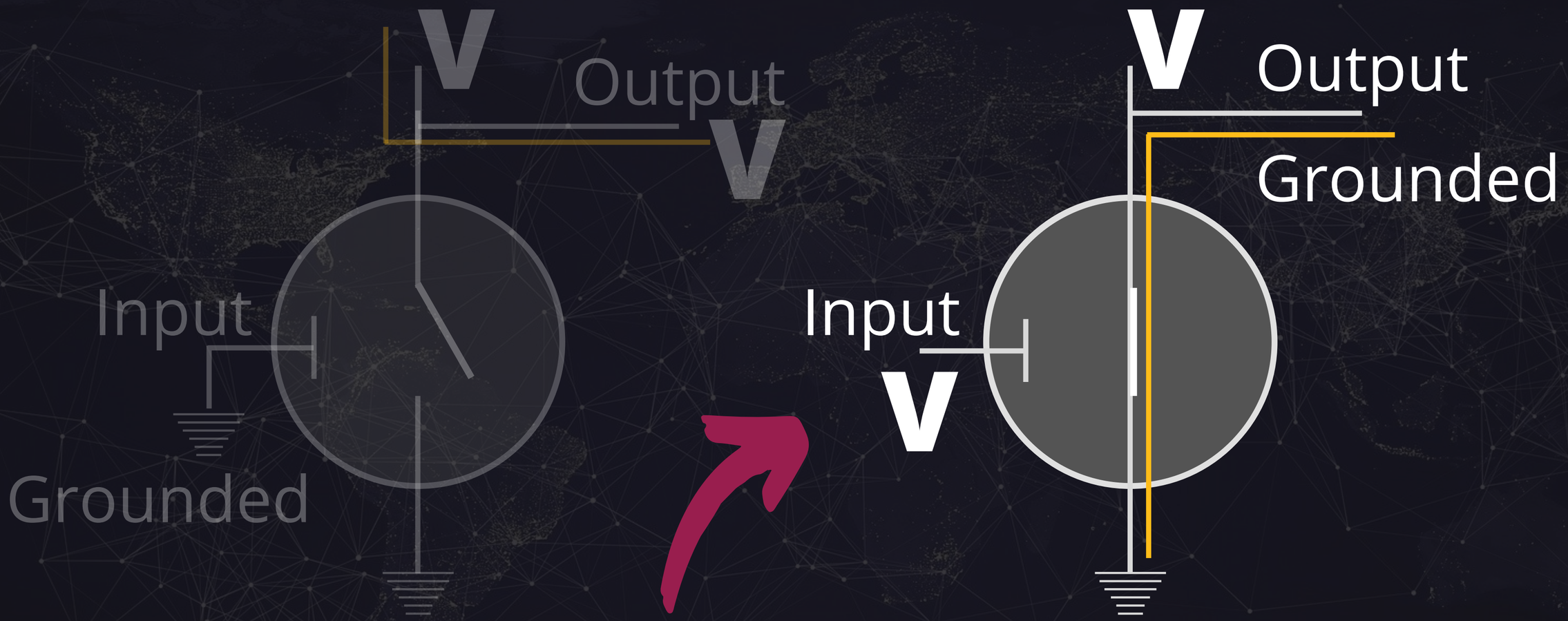
- A transistor has three connections: **Base** (B), **Emitter** (V_in), and **Collector** (V_out).
- The base acts as a switch. If the base is grounded, the emitter is not connected to the collector.
- If a voltage is applied, an electrical connection is enabled between emitter and collector.

Circuits

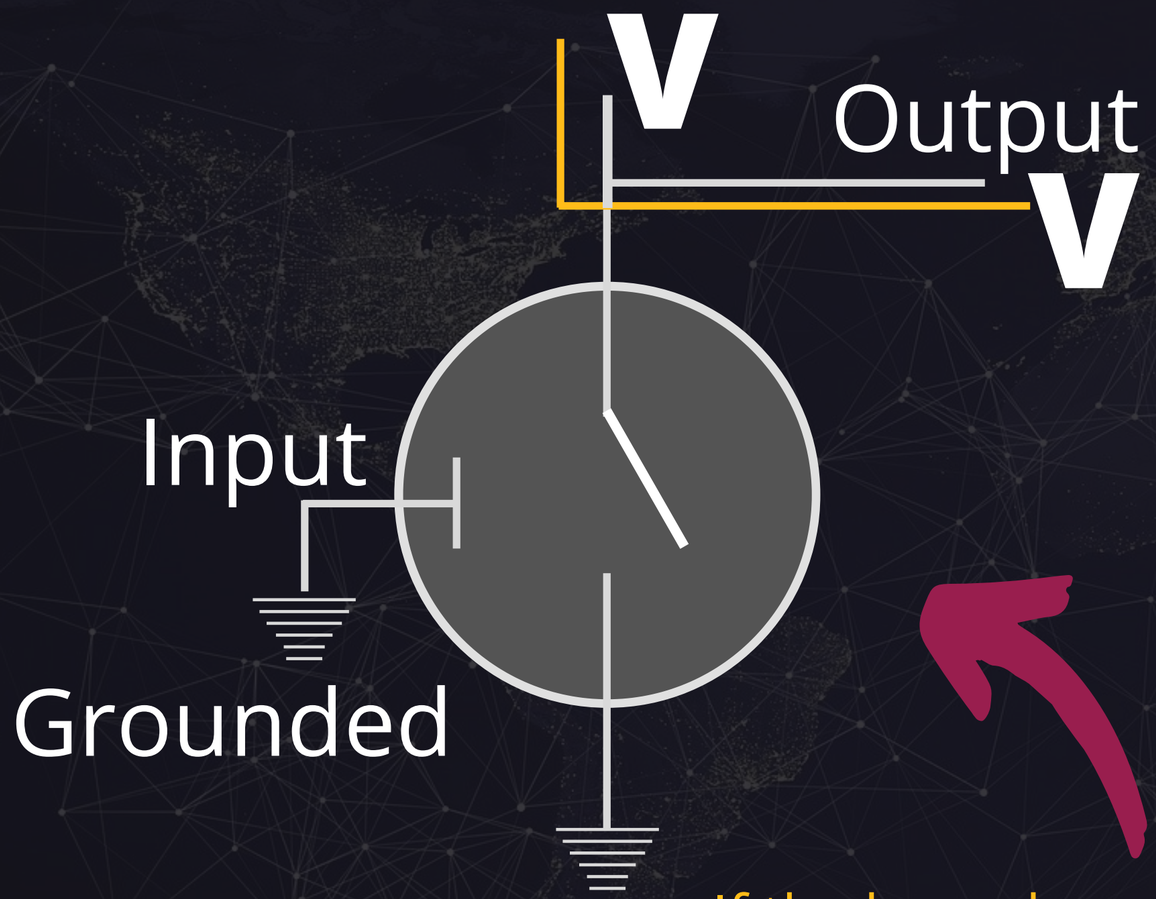
- Circuits are set up with a voltage source and opportunities for wires to be grounded.
- The circuits are set up with electrical resistances so if a wire is connected to ground, it is grounded, whether is is also connected to a voltage source or not.
- V_in is always connected to a source that might have a voltage on it or be grounded or be disconnected, while V_out may either be disconnected or grounded.



- V is a consistent voltage source.
- A connection of the output to voltage or ground is shown by a yellow path.

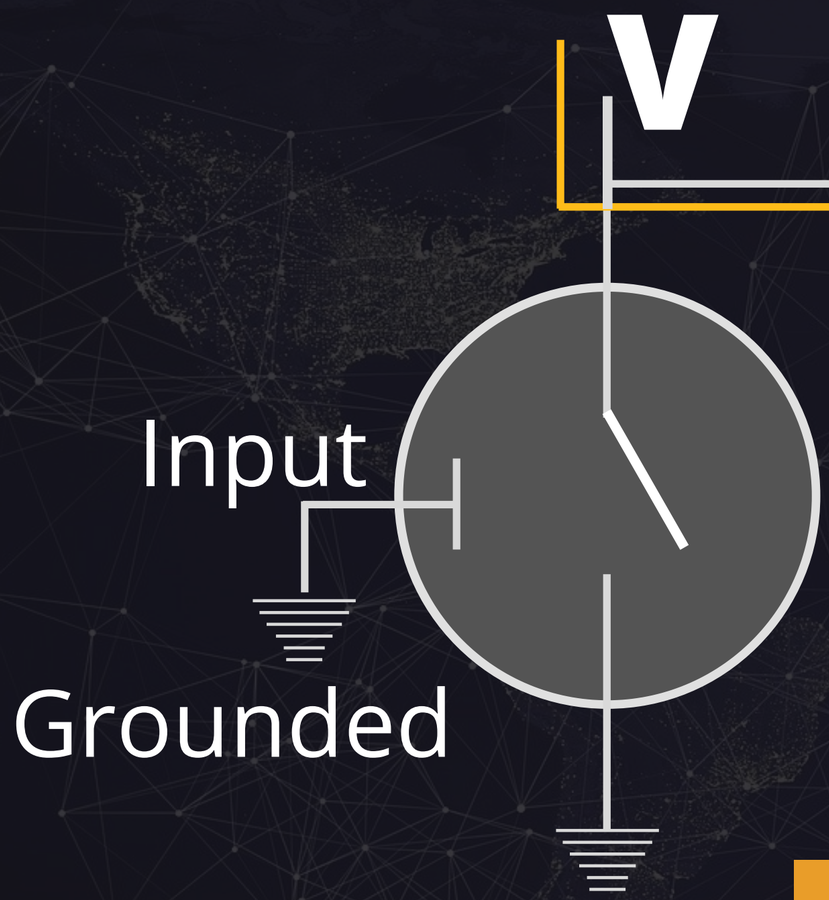


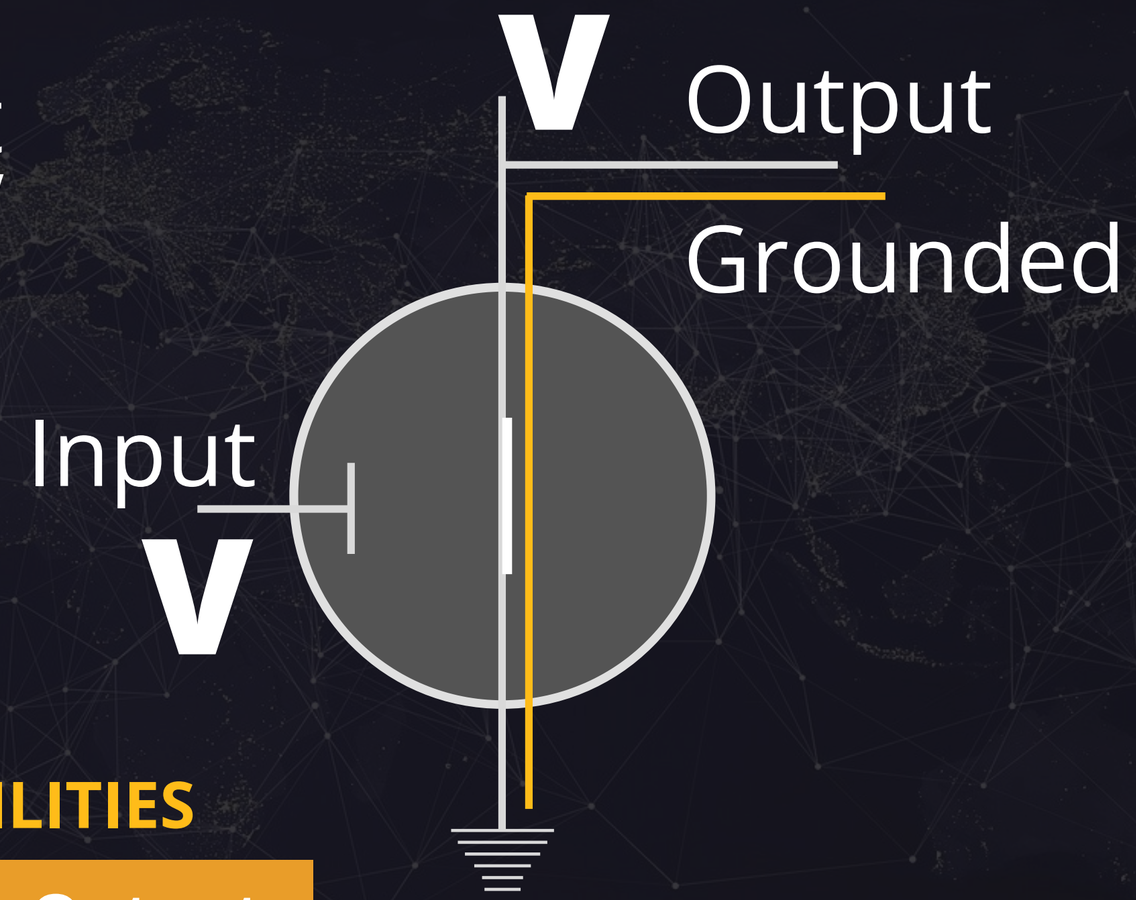
The base has a voltage, the emitter circuit is connected, and ground at V_out is connected to the output through V_in, so the output becomes grounded.





If the base does not have a voltage, the connection between the output and ground is broken, and the output is just connected to the regular voltage source, resulting in a voltage on the output wire.





POSSIBILITIES

Base	Output
Voltage	Grounded
Grounded	Voltage





POSSIBILITIES

Base	Output
Voltage	Grounded
Grounded	Voltage

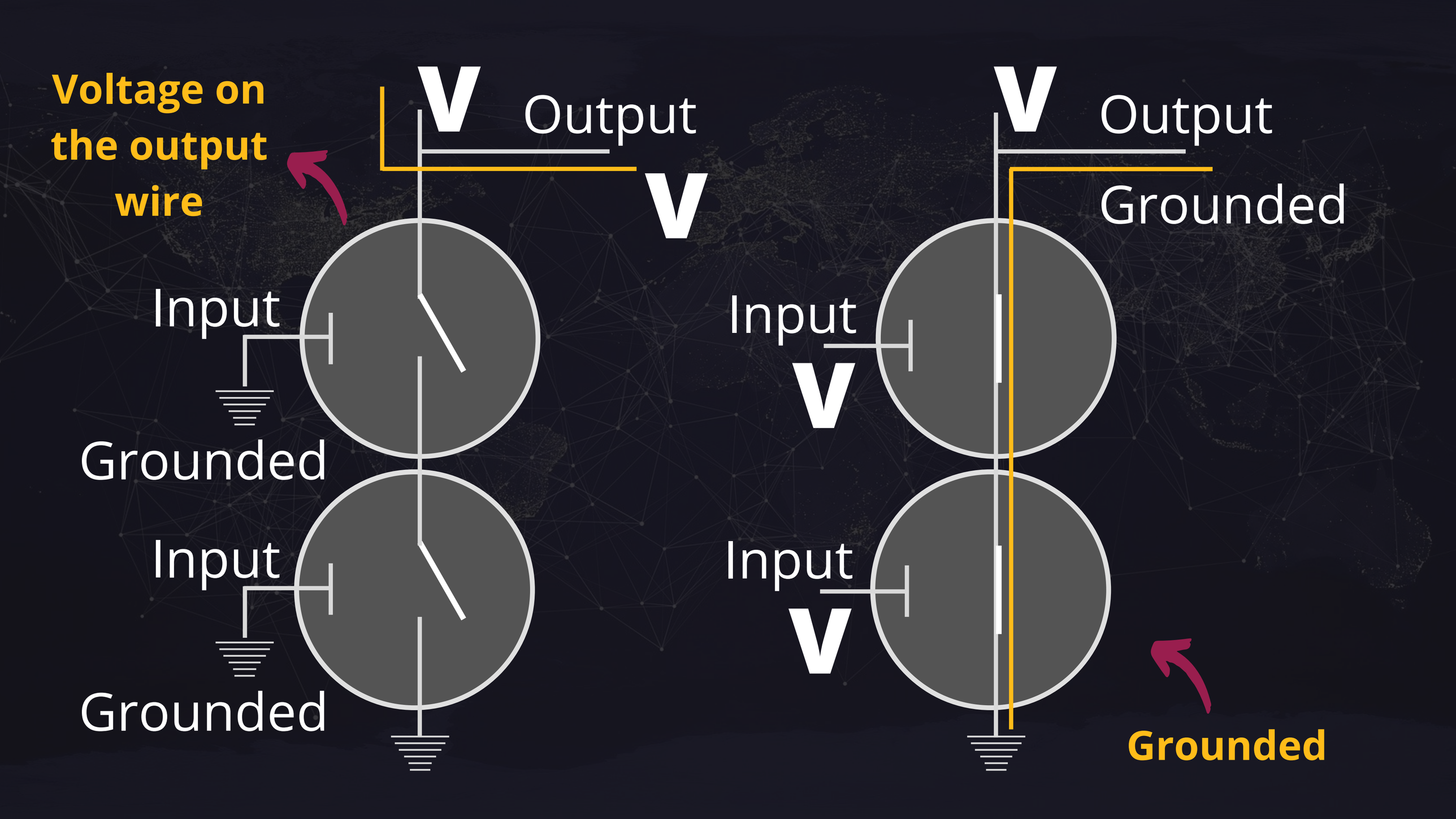
POSSIBILITIES

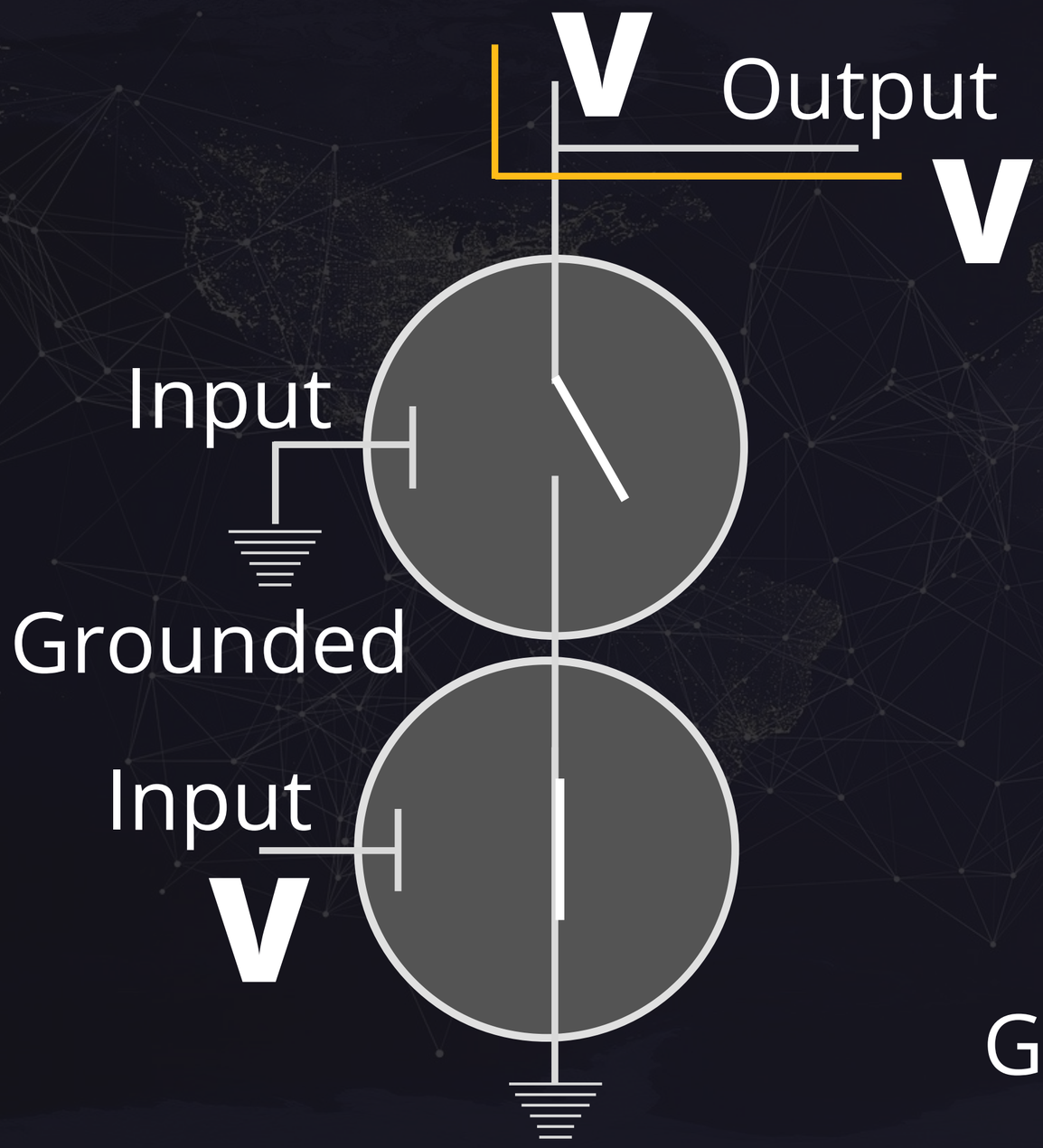
Base	Output
1	0
0	1

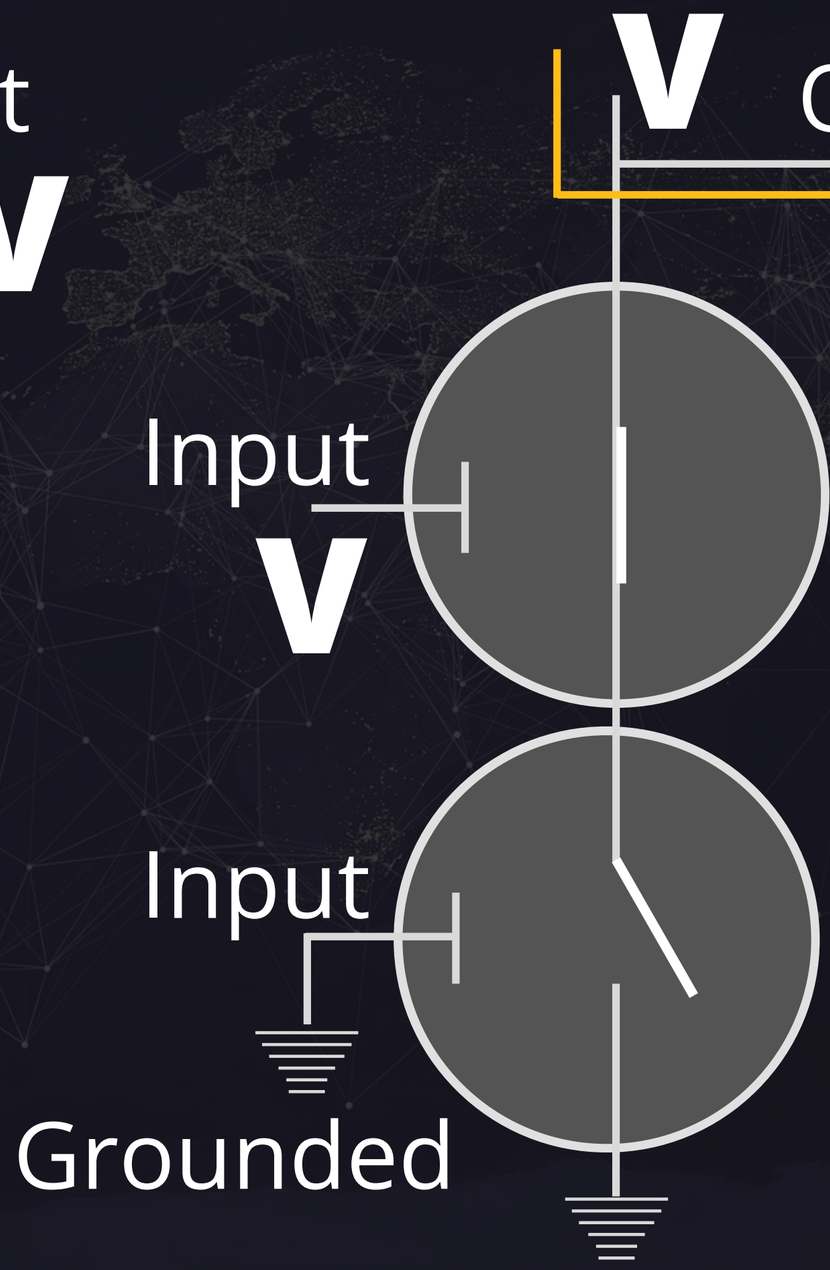
What **<u>truth table</u>** is this?

More transistors?

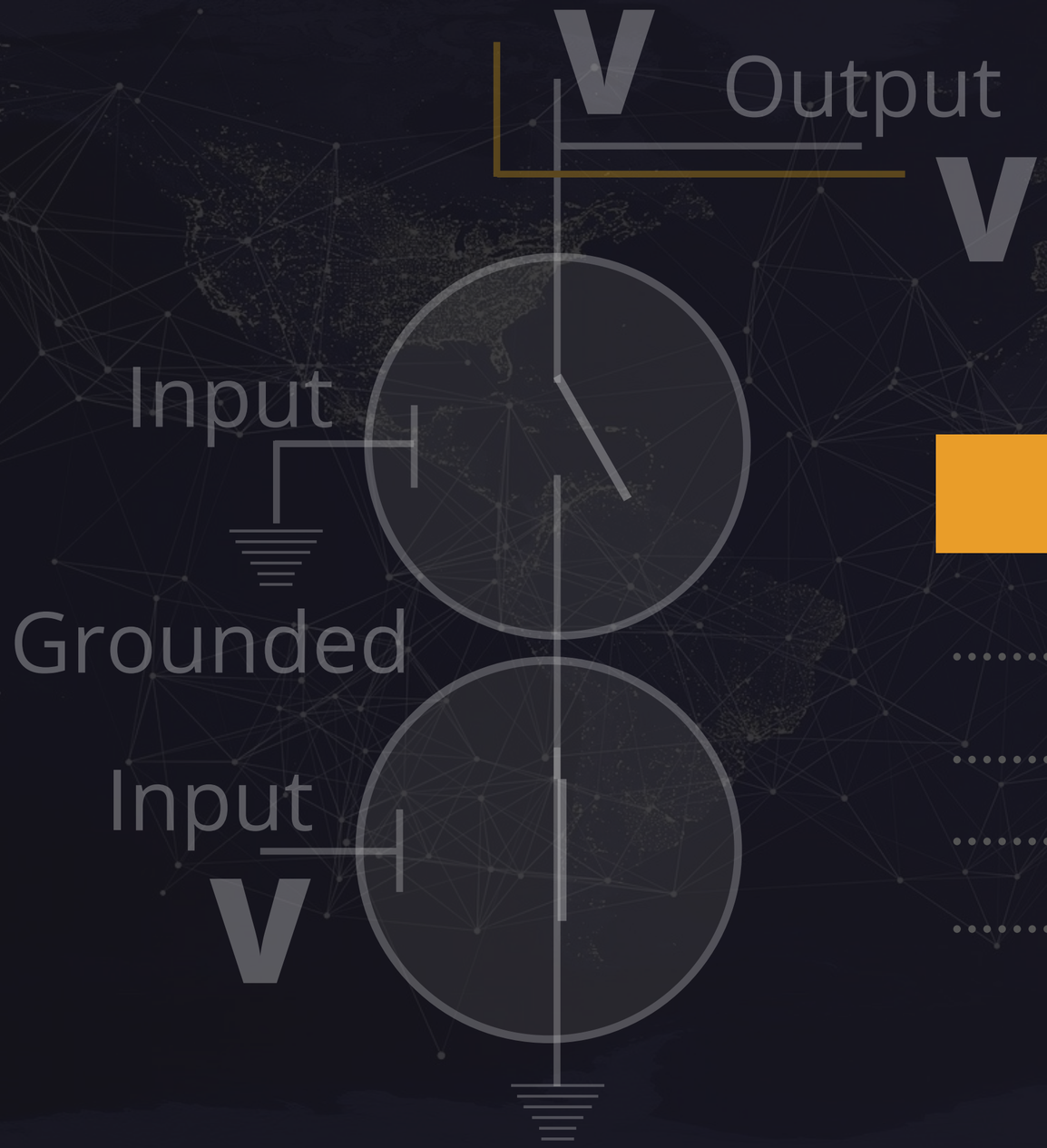
We can connect two transistors in **series** and use both bases as inputs.


Voltage on the output wire
V
Output
V
Input
Grounded
Input
Grounded
V
Output
Grounded
Input
V
Input
V
Grounded





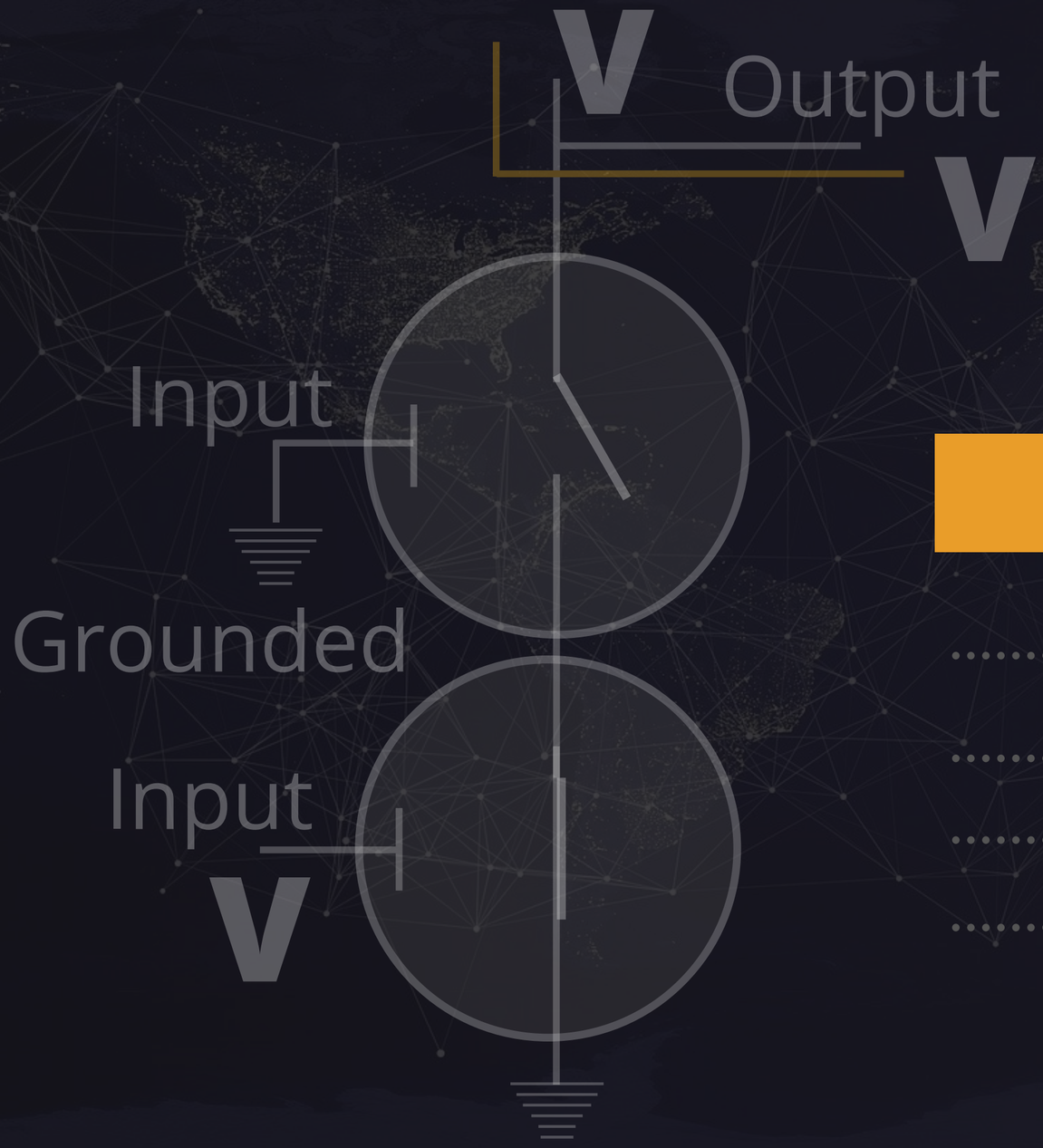
What are the possibilities now?



POSSIBILITIES

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

What **truth table** is this?



POSSIBILITIES

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0

NAND gate: Not AND

Exercise 4

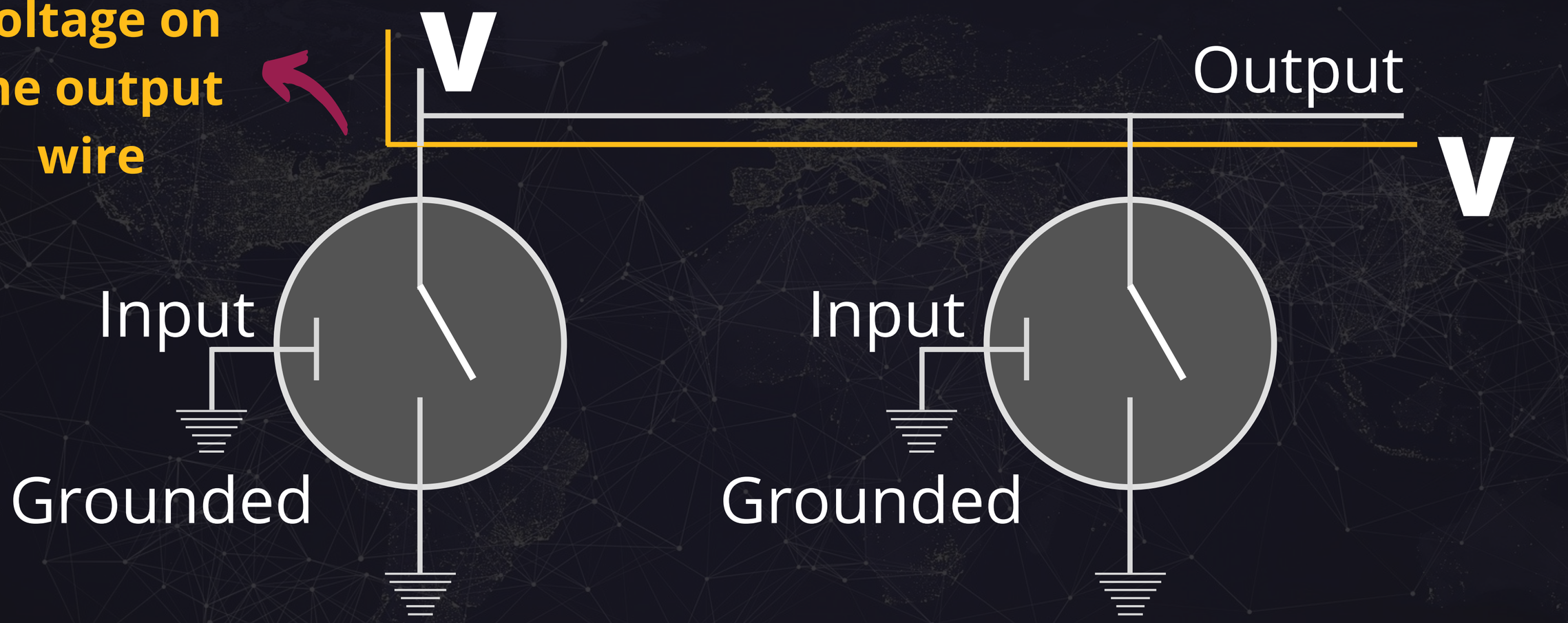
Write a Python program that produces the truth table for NAND (AB)'

05 MINUTES

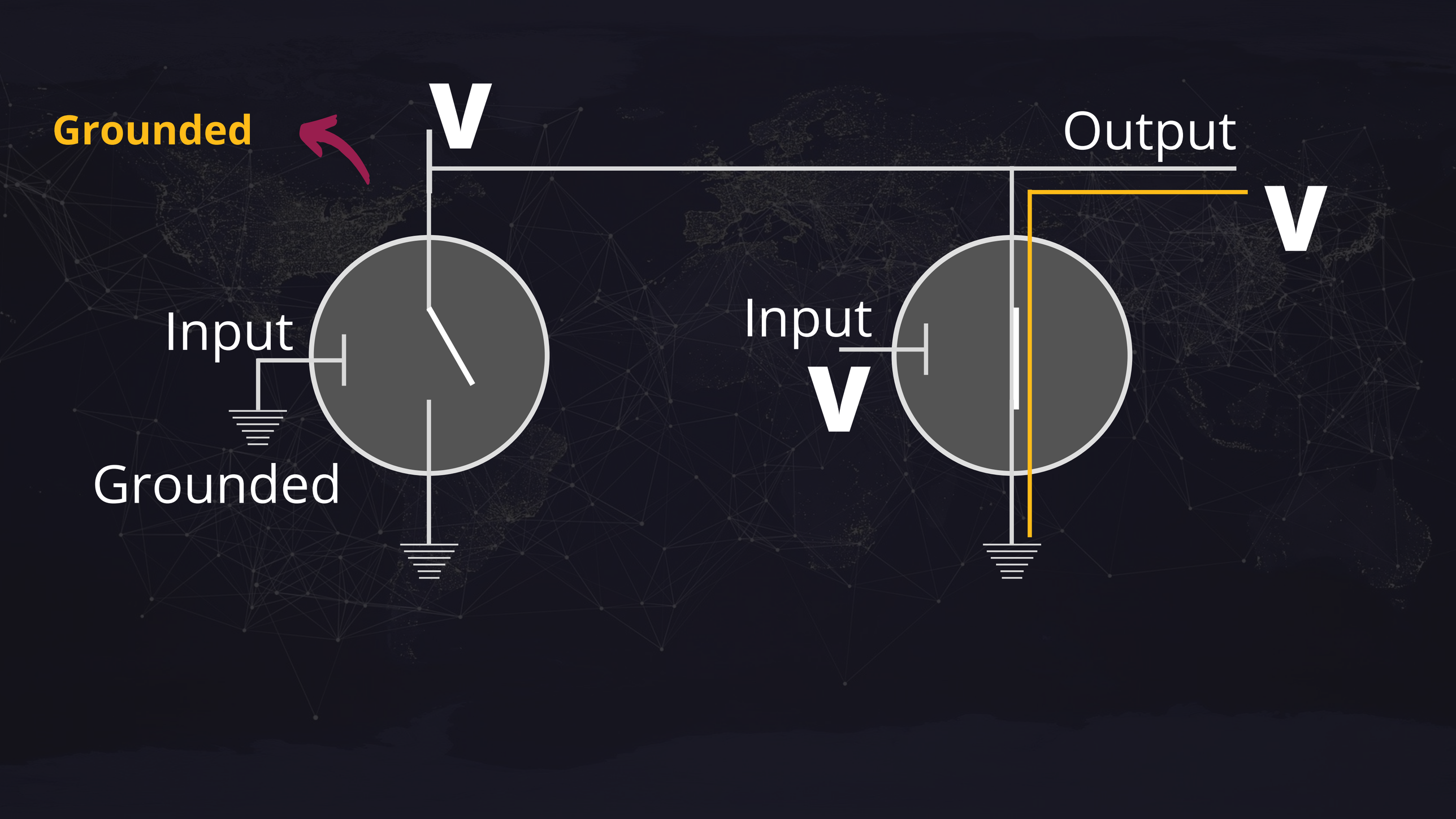


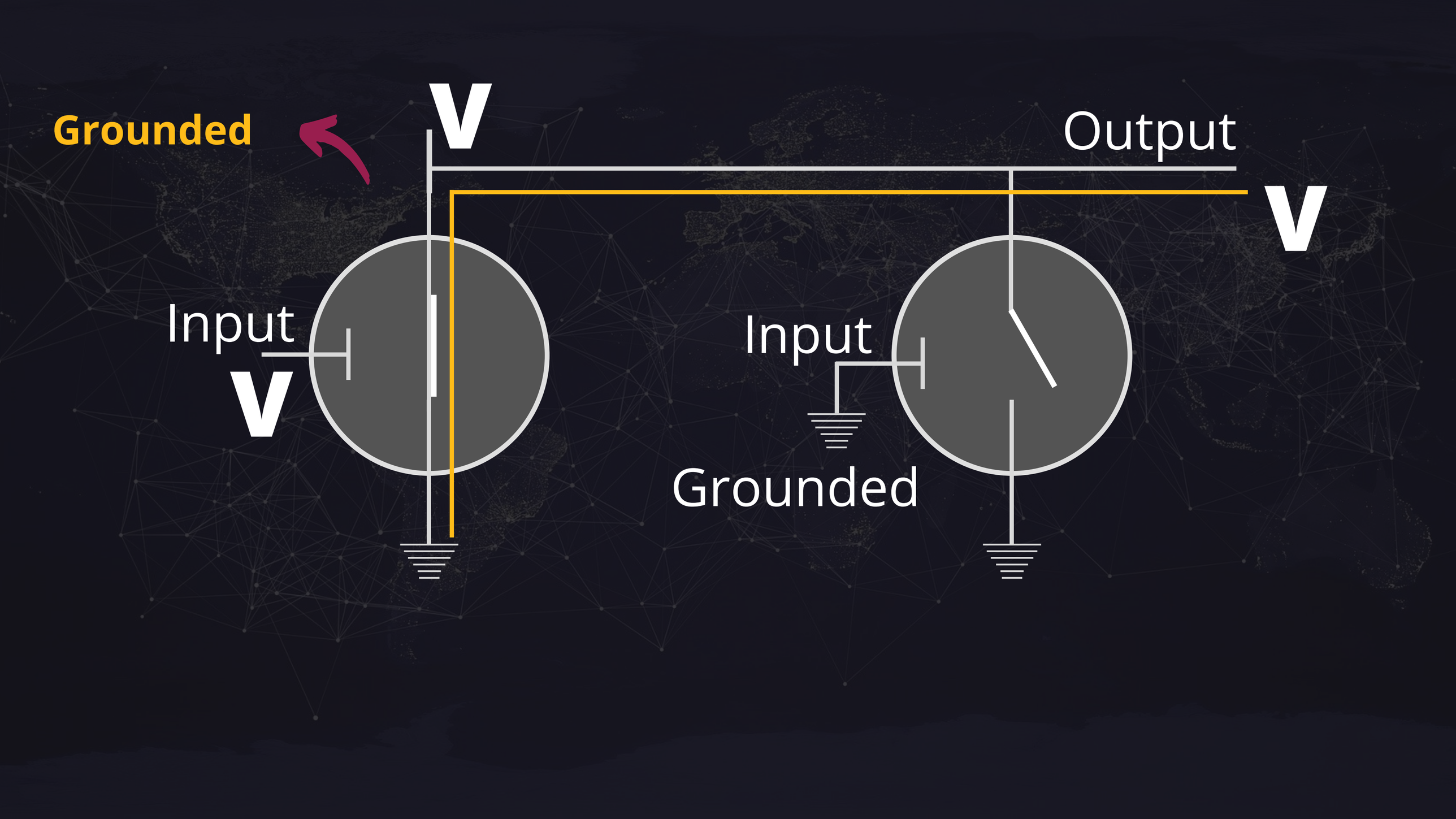
More transistors?

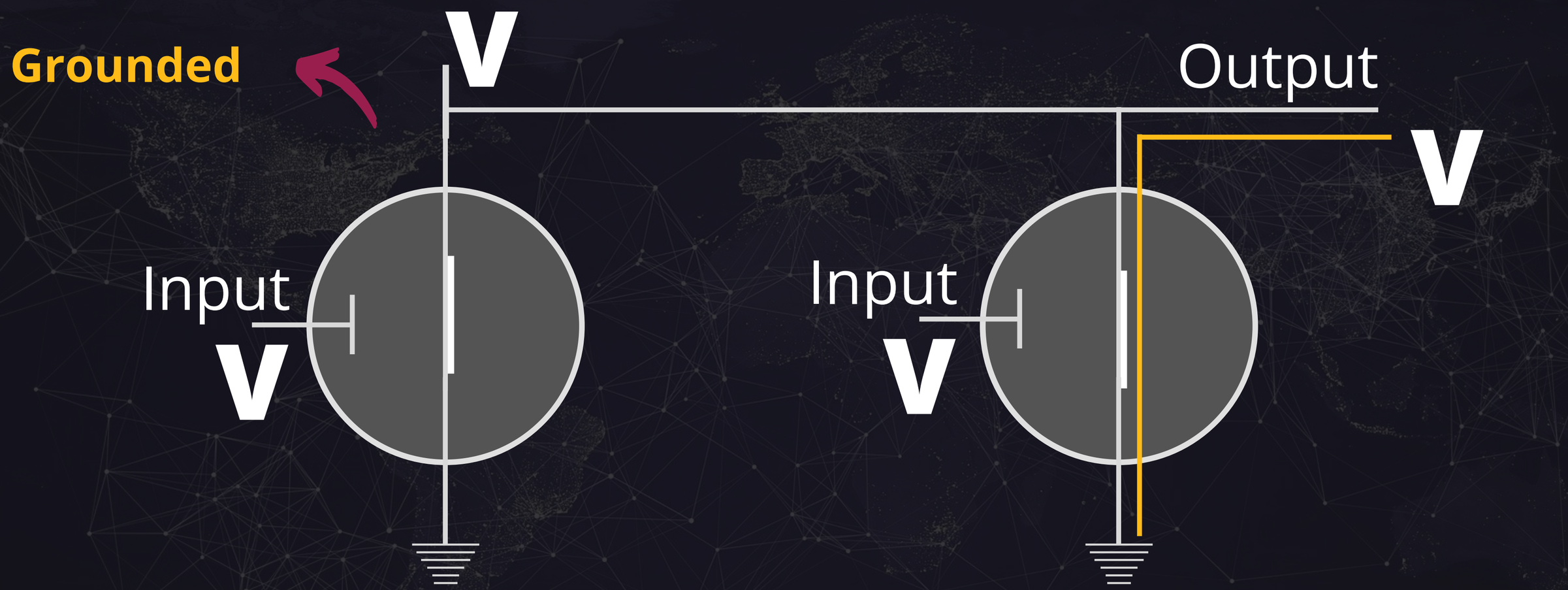
We can connect two transistors in **parallel** and use both bases as inputs.

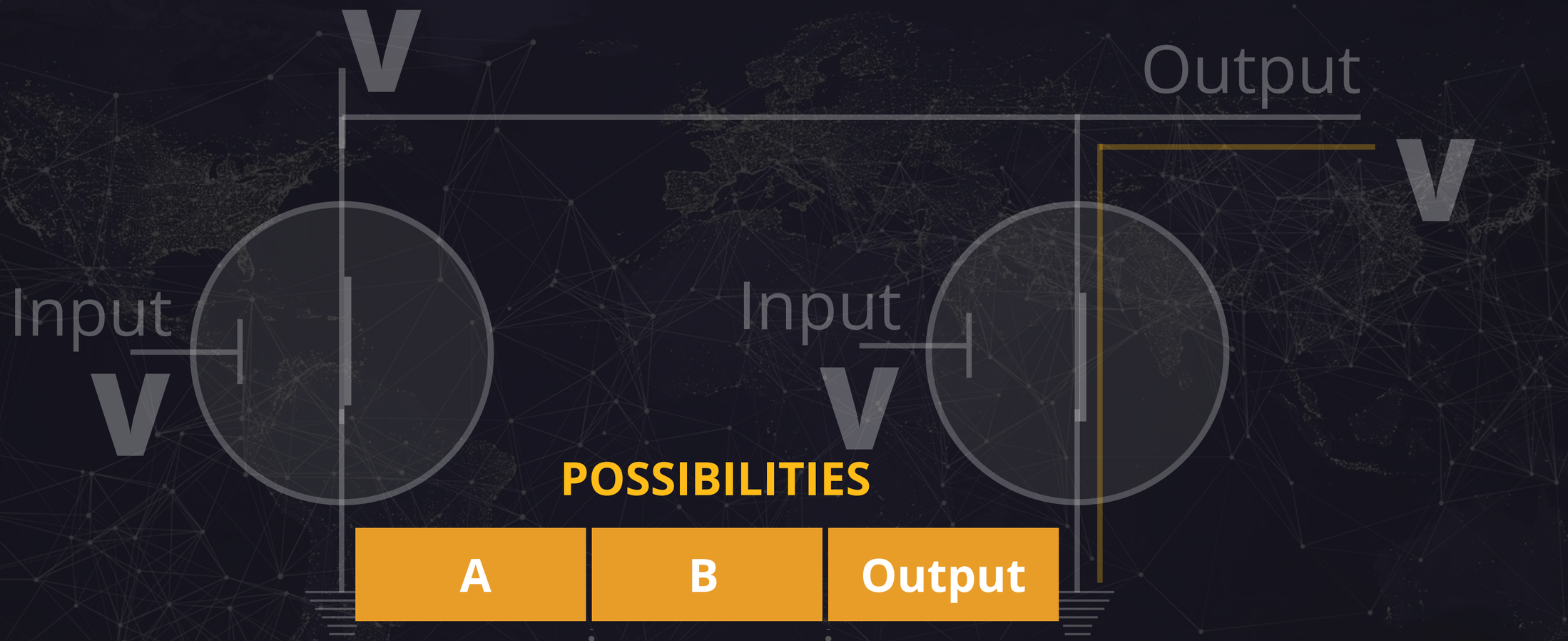


The only way to avoid a connection between the output and ground is to have both inputs grounded.


Grounded
V
Output
V
Input
Grounded
Input
V


Grounded
V
Output
V
Input
V
Input
Grounded

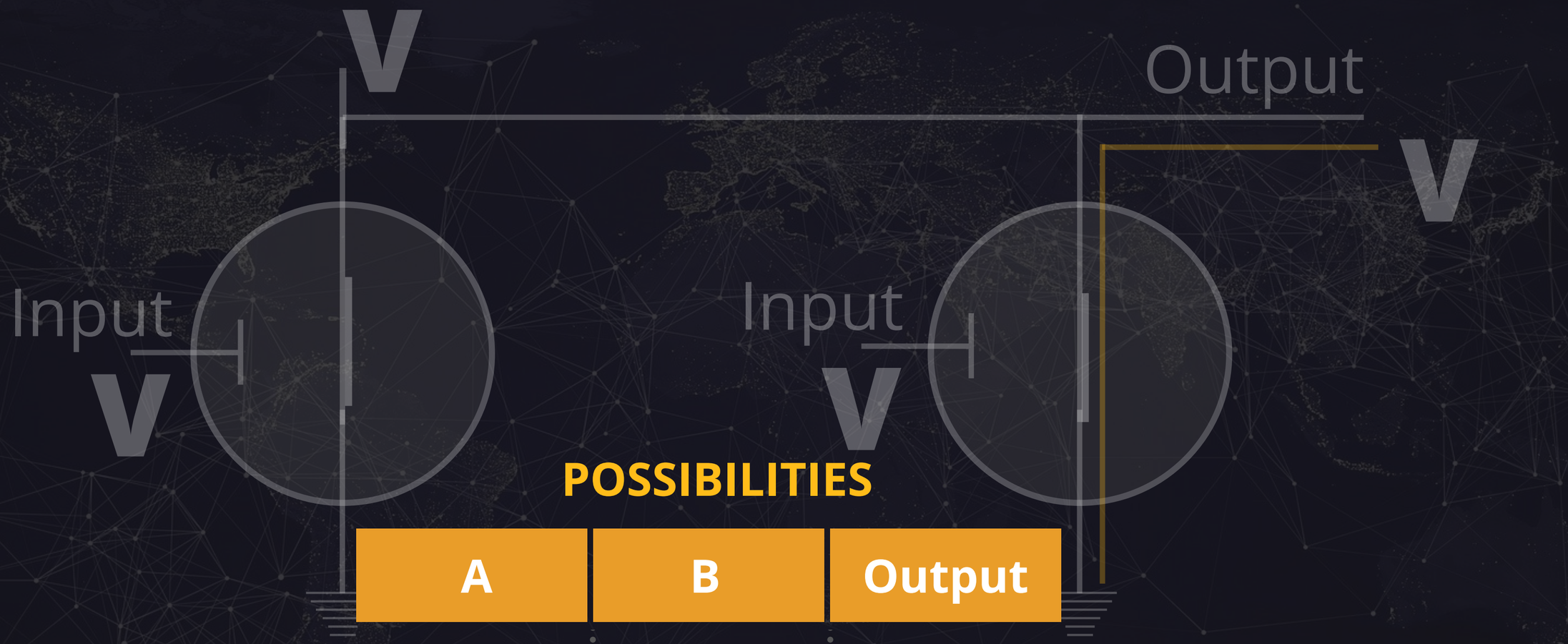

Grounded
V
Output
V
Input
V
Input
V



POSSIBILITIES

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

What **truth table** is this?



POSSIBILITIES

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0

NOR gate: Not OR

Exercise 4

Write a Python program that produces the truth table for NOR (A+B)'

05 MINUTES



Circuits

- **Not** gate needs one transistor.
- **NAND** gate needs two transistors.
- **NOR** gate needs two transistors.
- What about **AND** and **OR** ?
  - **AND** can be implemented by adding a transistor (**Not**) after the **NAND** gate.
    - ((AB)')' = AB
    - We need three transistors
  - **OR** can be implemented by adding a transistor (**Not**) after the **NOR** gate.
    - ((A+B)')' = A+B
    - We need three transistors.

GATES AND OPERATIONS

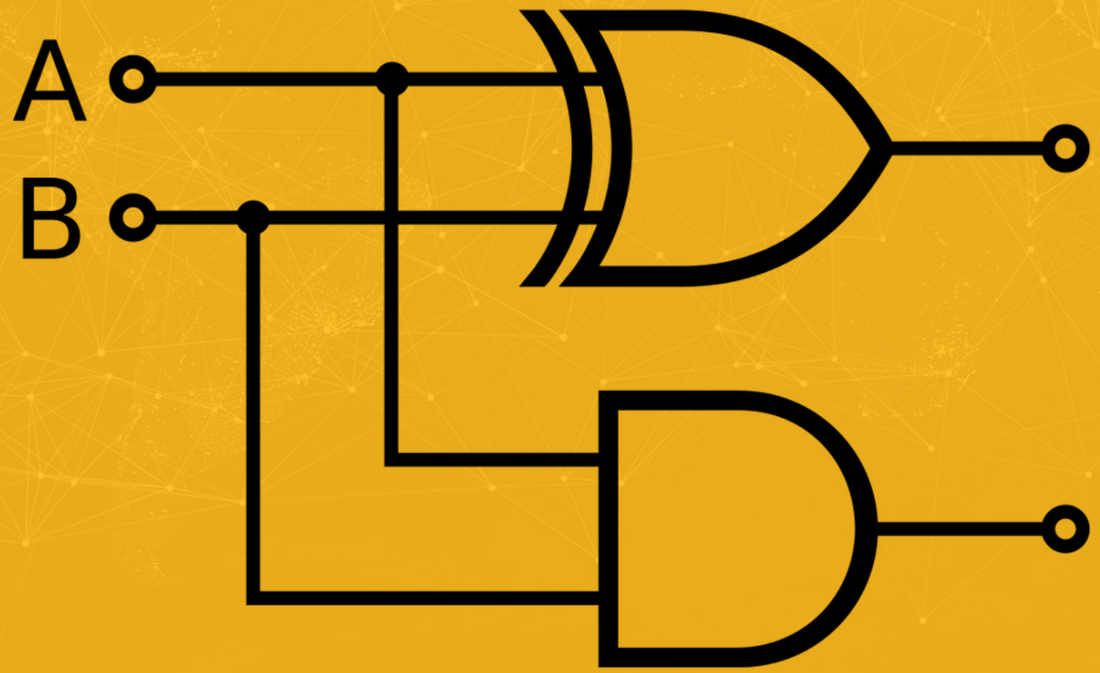
GATES

- In circuit diagrams, the details of the combinations of transistors are hidden
  - Astracted away.
- The operations are given special simple symbols.



GATES

- **Example 1:**

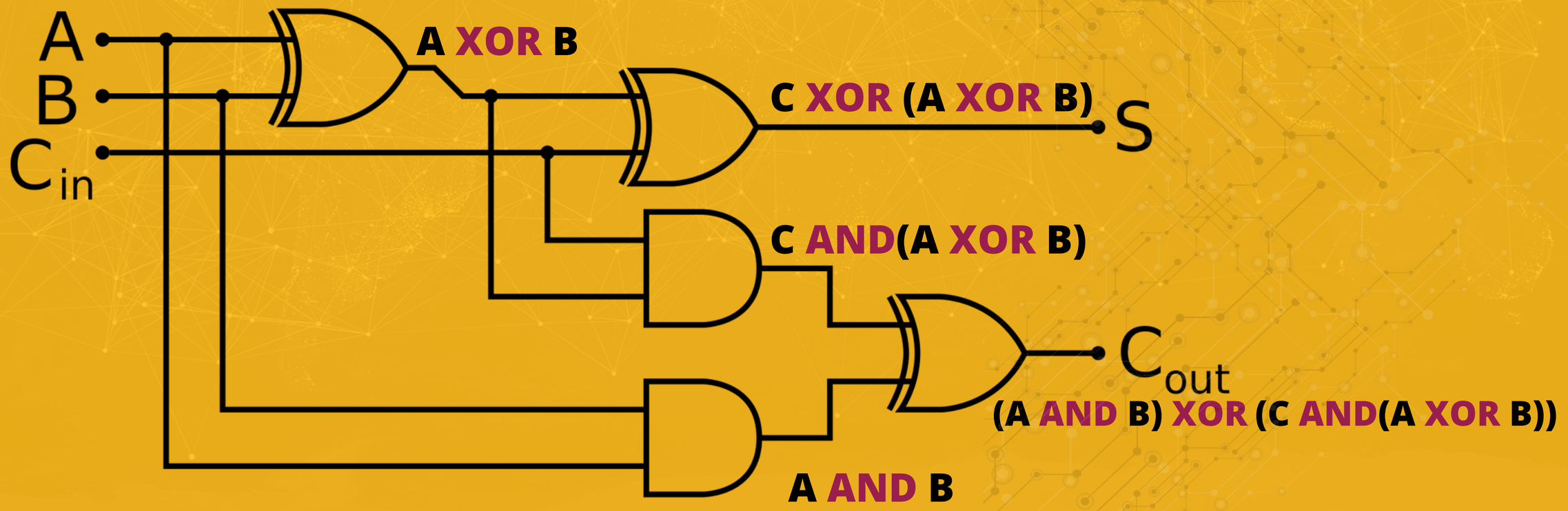


S = A XOR B

C = A AND B

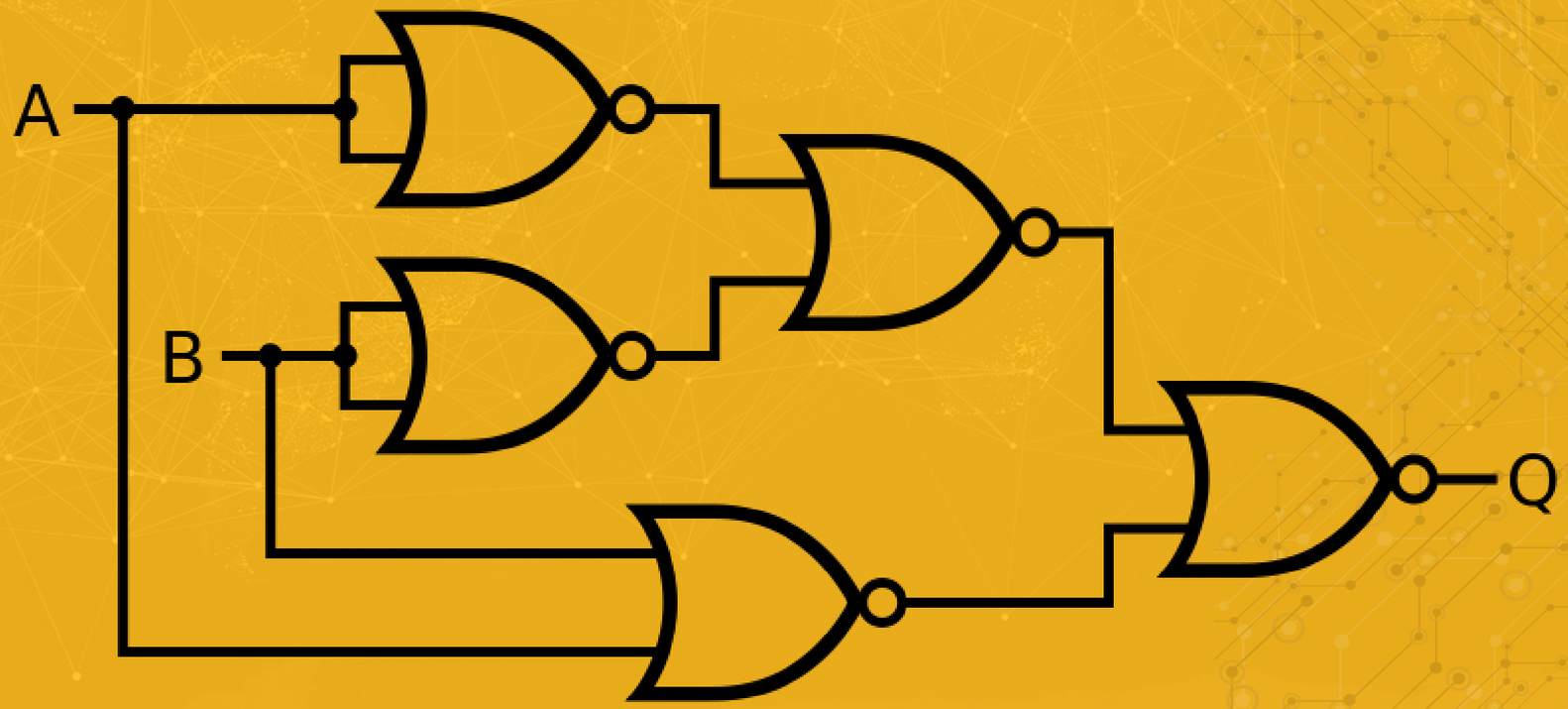
GATES

- **Example 2:**



Exercise 5

- **What is the Boolean expression for the following circuit diagram?**



Finding a Boolean Expression for a truth table

- What is the Boolean expression for the following truth table?

A	B	C	Output
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Finding a Boolean Expression for a truth table

- What is the Boolean expression for the following truth table?

A	B	C	Output
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1 ✓	1 ✓	1 ✓	1



ABC

Finding a Boolean Expression for a truth table

A	B	C	Output
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1 ✓	0 ✗	1 ✓	(1) ← AB'C
1	1	0	0
1 ✓	1 ✓	1 ✓	(1) ← ABC

Finding a Boolean Expression for a truth table

A	B	C	Output
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

A'B'C

AB'C

ABC

Finding a Boolean Expression for a truth table

A	B	C	Output
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

This truth table corresponds to the Boolean expression:

A'B'C + AB'C + ABC

Simplifying Boolean Algebra

- The Boolean expression from last example is:
  - A'B'C + AB'C + ABC
- Given the set of rules of Boolean Algebra, can we simplify the expression?

A'B'C + (AB'C + ABC)

A'B'C + AC (B' + B) ← Distributive rule

A'B'C + AC (1) ← Complement rule for OR

A'B'C + AC ← Identity rule

Simplifying Boolean Algebra

- The Boolean expression from last example is:
  - A'B'C + AB'C + ABC
- Given the set of rules of Boolean Algebra, can we simplify the expression?

A'B'C + (AB'C + ABC)

A'B'C + AC (B' + B) ← Distributive rule

A'B'C + AC (1) ← Complement rule for OR

A'B'C + AC ← Identity rule

IS there another way?

Simplifying Boolean Algebra

A'B'C + AB'C + ABC	
A'B'C + AB'C + AB'C + ABC	← Adding a second ORed copy of AB'C
(A'B'C + AB'C) + (AB'C + ABC)	
(A'B'C + AB'C) + AC (B'+B)	← Distributive rule
(A'B'C + AB'C) + AC	← Identity rule
(A' + A) B'C + AC	← Distributive rule
B'C + AC	← Identity rule

Exercise 6

- What is the Boolean expression for the following truth table?

A	B	C	Output
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Arithmetic Operations in Computer Circuits

Arithmetic Operations in Computer Circuits

- So far, we can convert logical operations to hardware.
  - AND, NAND, OR, NOR, XOR, XNOR, NOT
- Now, how to do **normal arithmetic**?

Addition in Computer Circuits

- Let's start with the simplest example
  - 1-bit addition.

$$\begin{array}{r} 0 \\ +\ 0 \\ \hline 0 \end{array} \qquad \begin{array}{r} 0 \\ +\ 1 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 0 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 1 \\ \hline 10 \end{array}$$



We need to keep track of two binary outputs to account for the **carry bit**

Addition in Computer Circuits

$$\begin{array}{r} 0 \\ +\ 0 \\ \hline 0 \end{array} \qquad \begin{array}{r} 0 \\ +\ 1 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 0 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\ 1 \\ \hline 10 \end{array}$$

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Addition in Computer Circuits

0	0	1	1
+ 0	+ 1	+ 0	+ 1
0	1	1	10

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

What gate?

Addition in Computer Circuits

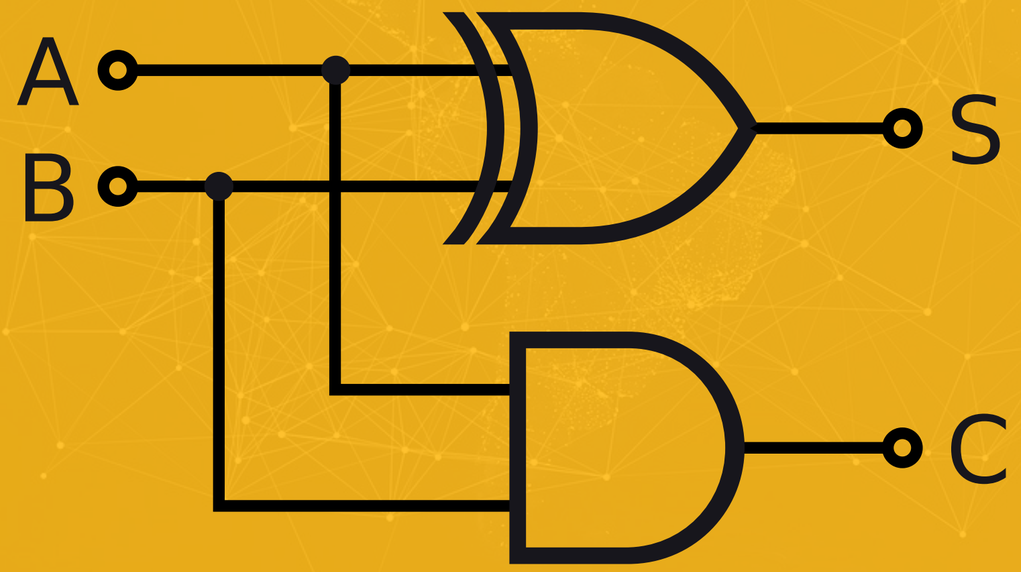
$$\begin{array}{r} 0 \\ +\,0 \\ \hline 0 \end{array} \qquad \begin{array}{r} 0 \\ +\,1 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\,0 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +\,1 \\ \hline 10 \end{array}$$

XOR

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

What gate?

Addition in Computer Circuits



HALF ADDER CIRCUIT

AND XOR

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Addition in Computer Circuits

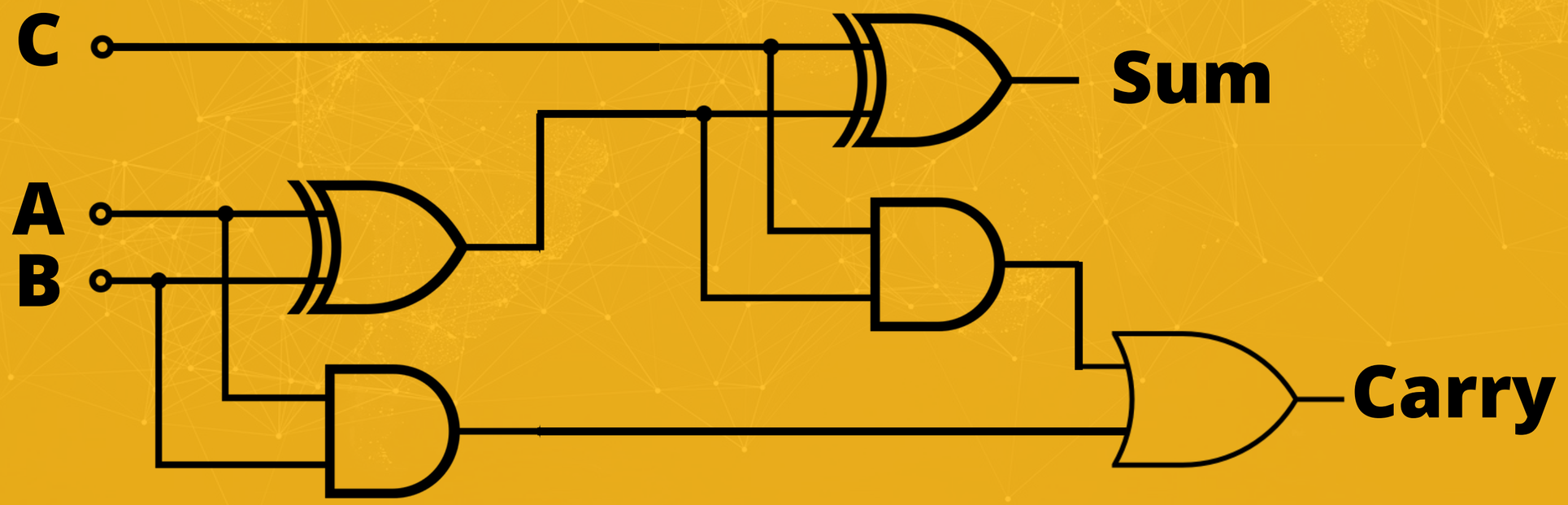
What about adding multi-bit binary numerals?



We need two half adders and one OR.
This creates a **full adder**.

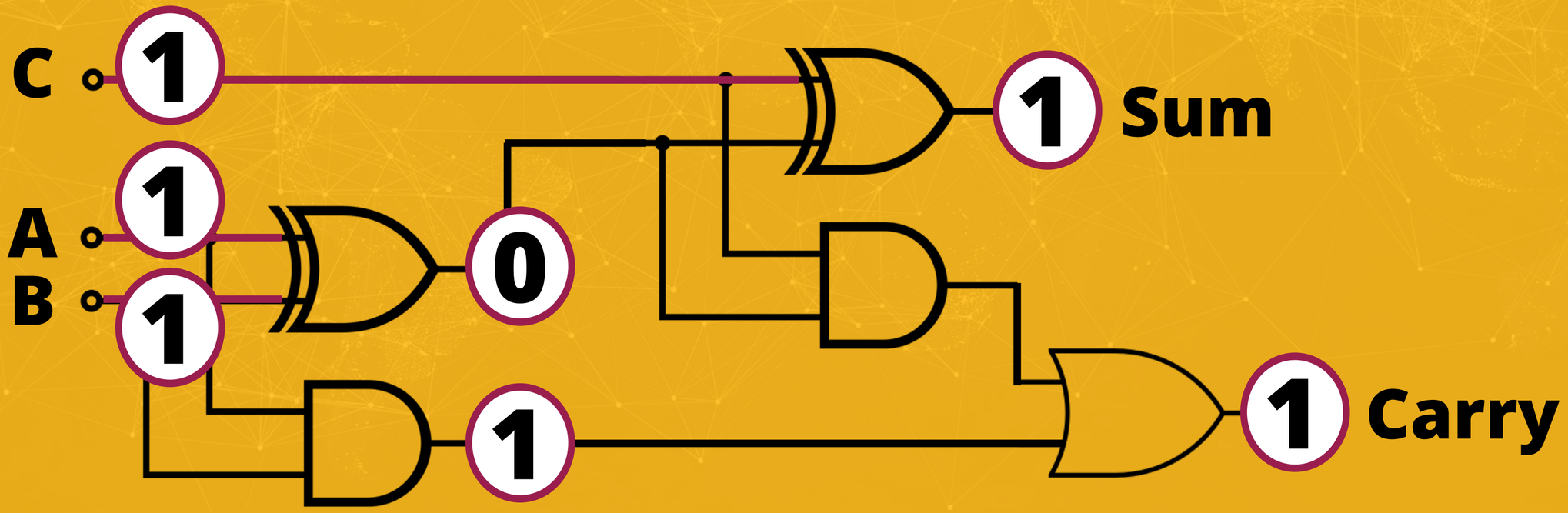
Addition in Computer Circuits

Full adder



Addition in Computer Circuits

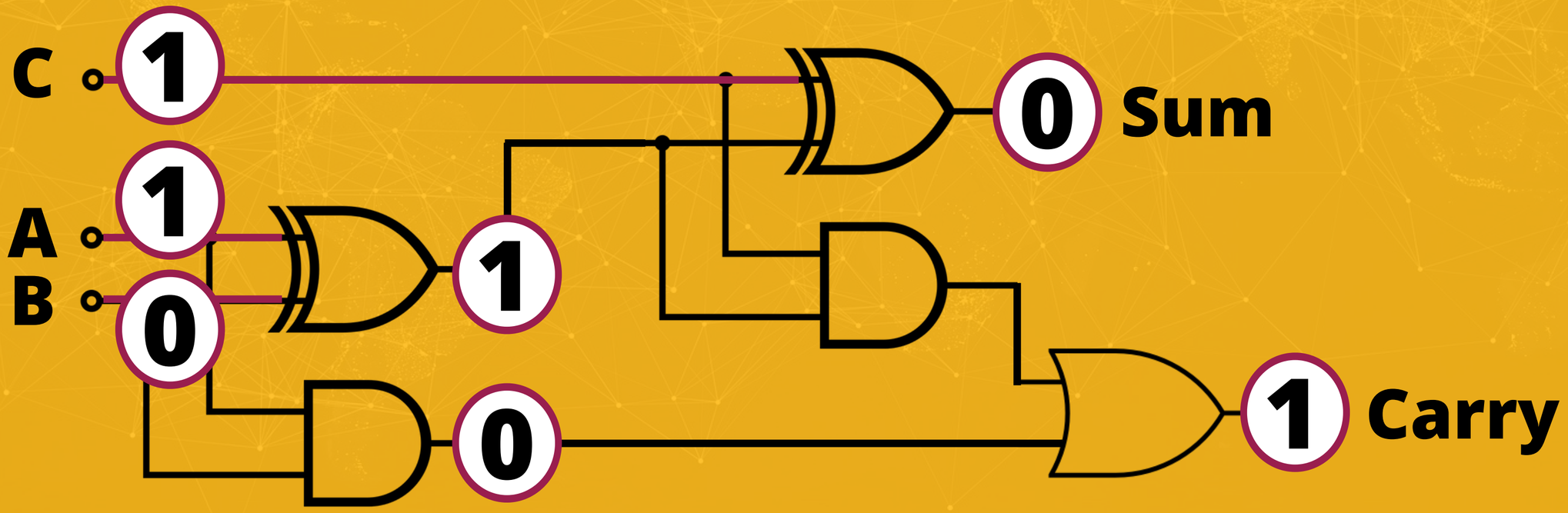
Example:



1+1+1 = 11

Addition in Computer Circuits

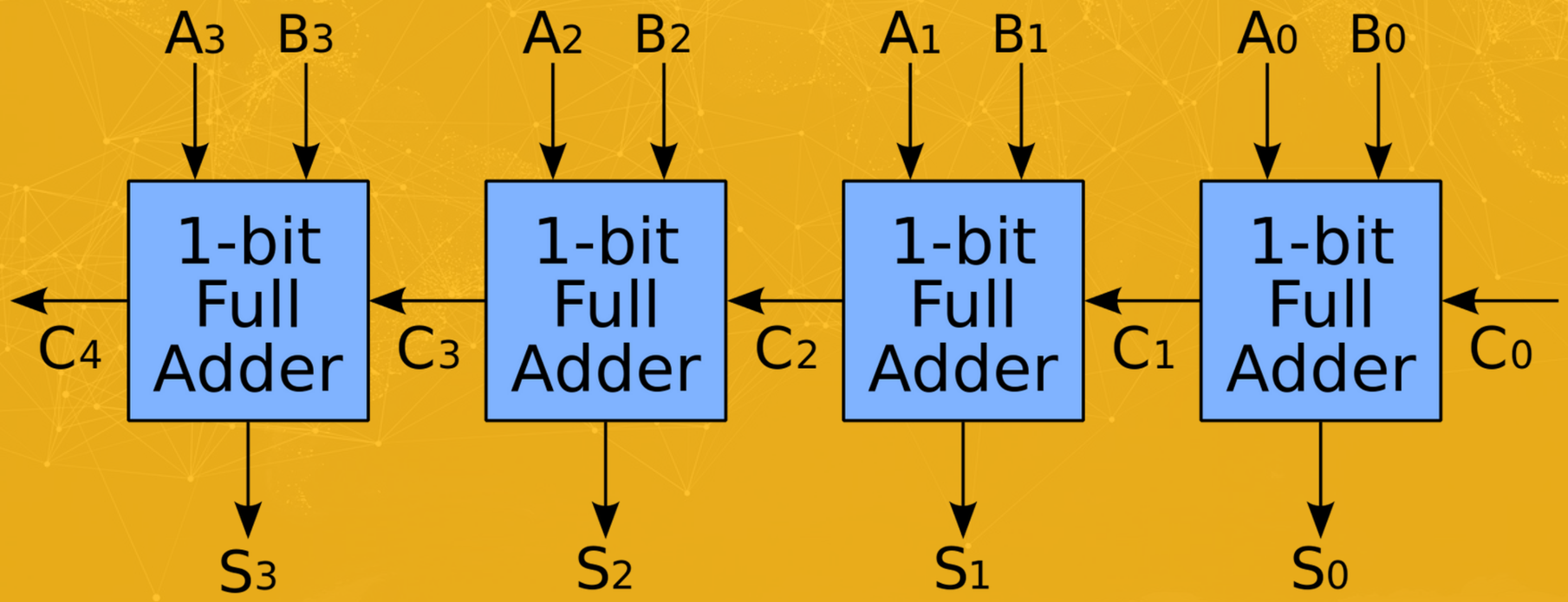
Example:



1+0+1 = 10

Addition in Computer Circuits

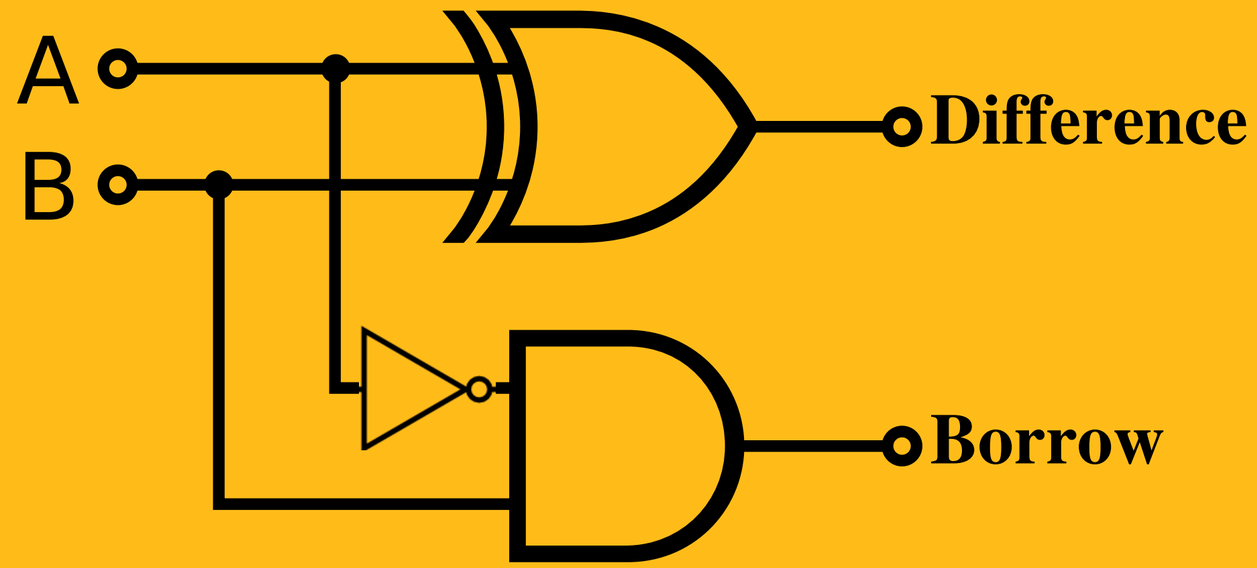
We can chain full adders to create a multi-bit adder.



Subtraction in Computer Circuits

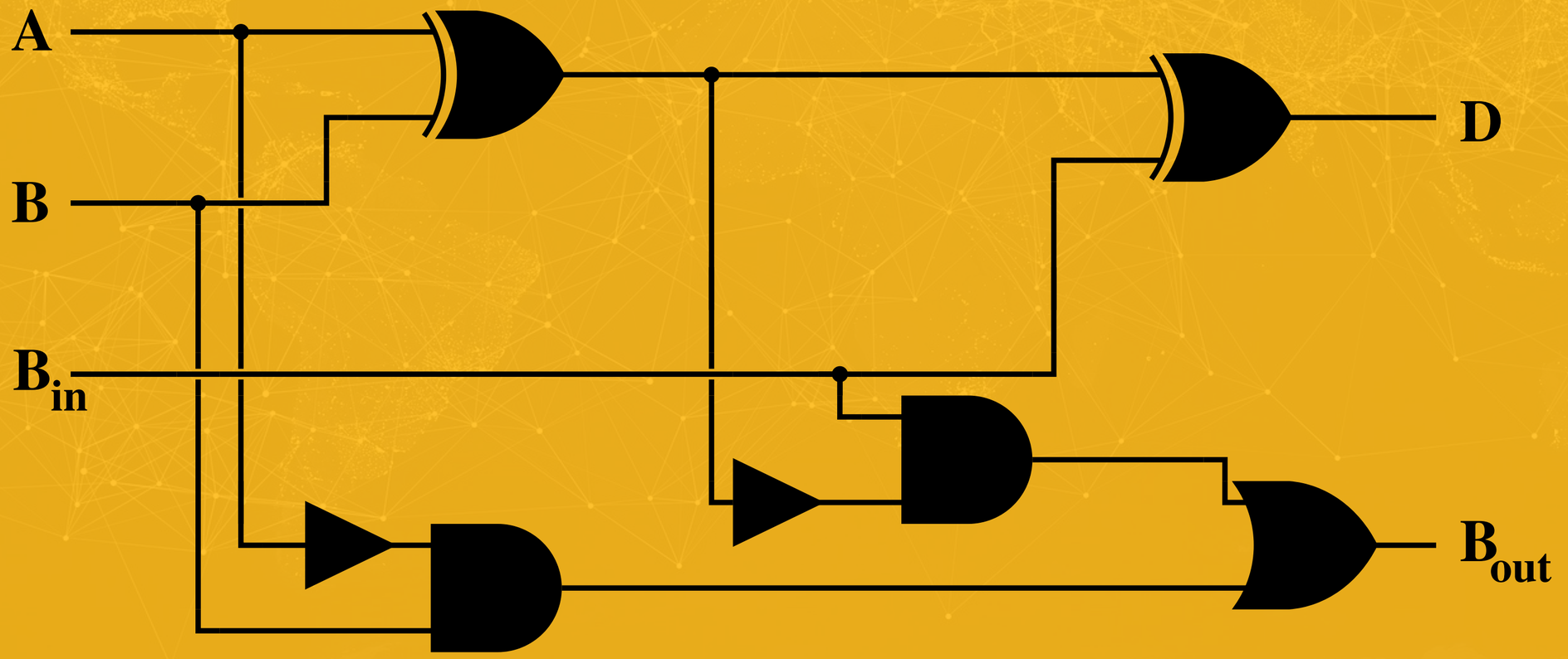
Other operations can be done with variations of addition operation and manipulations of bits.

Subtraction is achieved usually by using 2's complement addition



HALF SUBTRACTOR (A - B)

Subtraction in Computer Circuits



FULL SUBTRACTOR (A - B)

Exercise 7

- How to use the full subtractor for multi-bit subtractor?
- How to use adders for multiplication?

END PART 2